

上下文硬盘缓存

DeepSeek API 上下文硬盘缓存技术对所有用户默认开启，用户无需修改代码即可享用。

用户的每一个请求都会触发硬盘缓存的构建。若后续请求与之前的请求在前缀上存在重复，则重复部分只需要从缓存中拉取，计入“缓存命中”。

缓存落盘与命中规则

缓存命中的前提是相应前缀已被“落盘”（写入硬盘缓存）。受 Sliding Window Attention 机制的影响，缓存前缀的存取与判别与之前有所不同。每条缓存前缀是一个独立的完整单元。后续请求只有在完整匹配缓存前缀单元时，才能命中缓存。

缓存前缀落盘时机：

- 请求结束位置落盘：**每次请求的用户输入结束位置与模型输出结束位置，会产生两个缓存前缀单元。后续请求若完整匹配了它们，则可命中。
- 公共前缀检测落盘：**当系统检测到多次请求之间存在公共前缀时，会将该公共前缀作为一个独立的缓存前缀单元进行落盘。后续请求若完整复用了该缓存前缀单元，则可命中。
- 按固定 token 间隔落盘：**在长输入或长输出中，系统会以一定的 token 数量为间隔，截取缓存前缀单元，避免长前缀因迟迟未达到结束位置而完全无法被缓存。

举例 1：用户第一轮请求内容为 `A + B`，第二轮请求内容为 `A + B + C`，则第二轮请求能完整匹配 `A + B` 这个缓存前缀单元，可以命中 `A + B` 的缓存。详见下文例一。

举例 2：用户第一轮请求的内容为 `A + B`，第二轮请求的内容为 `A + C`，则第二轮请求无法命中缓存，因为 `A + C` 不能完整匹配第一轮缓存前缀单元（`A + B`）。但此时系统会识别到两轮请求存在公共前缀 `A`，并将 `A` 作为缓存前缀单元落盘。当第三轮请求 `A + D` 到来时，能完整匹配 `A` 这个缓存前缀单元，可以命中 `A` 的缓存。详见下文例二。

例一：多轮对话

第一次请求

```
messages: [
  {"role": "system", "content": "你是一位乐于助人的助手"},
  {"role": "user", "content": "中国的首都是哪里? "}
]
```

第二次请求

```
messages: [
  {"role": "system", "content": "你是一位乐于助人的助手"},
  {"role": "user", "content": "中国的首都是哪里? "},
  {"role": "assistant", "content": "中国的首都是北京。"},
  {"role": "user", "content": "美国的首都是哪里? "}
]
```

在上例中，第二次请求可以完整复用第一次请求的**缓存前缀单元**，这部分会计入“缓存命中”。

例二：长文本问答

第一次请求

```
messages: [
  {"role": "system", "content": "你是一位资深的财报分析师..."},
  {"role": "user", "content": "<财报内容>\n\n请总结一下这份财报的关键信息。"}
]
```

第二次请求

```
messages: [
  {"role": "system", "content": "你是一位资深的财报分析师..."},
  {"role": "user", "content": "<财报内容>\n\n请分析一下这份财报的盈利情况。"}
]
```

第三次请求

```
messages: [
  {"role": "system", "content": "你是一位资深的财报分析师..."}
```

```
{ "role": "user", "content": "<财报内容>\n\n请分析一下公司收入与支出占比。" }
]
```

在上例中，前两次请求不会命中缓存。前两次请求完成后，系统会识别出 `system` 消息 + `user` 消息中的<财报内容>为**缓存前缀单元**，并进行落盘。在第三次请求中，由于完整匹配了前面落盘的**缓存前缀单元**，则可命中缓存。

查看缓存命中情况

在 DeepSeek API 的返回中，我们在 `usage` 字段中增加了两个字段，来反映请求的缓存命中情况：

1. `prompt_cache_hit_tokens`：本次请求的输入中，缓存命中的 tokens 数
2. `prompt_cache_miss_tokens`：本次请求的输入中，缓存未命中的 tokens 数

硬盘缓存与输出随机性

硬盘缓存只匹配到用户输入的前缀部分，输出仍然是通过计算推理得到的，仍然受到 `temperature` 等参数的影响，从而引入随机性。其输出效果与不使用硬盘缓存相同。

其它说明

1. 缓存系统是“尽力而为”，不保证 100% 缓存命中
2. 缓存构建耗时为秒级。缓存不再使用后会自动被清空，时间一般为几个小时到几天